
1. REACTIVE AGENT-BASED TRAFFIC CONTROL SYSTEM

Introduction

A **Reactive Agent** is an AI agent that responds directly to changes in its environment without maintaining complex long-term reasoning.

In a smart traffic control system, the agent continuously receives real-time traffic information and immediately changes traffic signals to reduce congestion.

Objective

To design a Reactive Agent that:

- Monitors real-time traffic.
- Detects congestion.
- Dynamically changes traffic signals.
- Reduces waiting time.
- Processes streaming traffic data.

System Architecture

Traffic Sensors / Cameras



Streaming Data



Data Processing



Reactive Agent



Traffic Condition



Signal Control Decision



Traffic Lights



Reduced Congestion

Inputs to the Agent

The agent receives real-time information such as:

- Number of vehicles.
 - Vehicle speed.
 - Queue length.
 - Lane occupancy.
 - Emergency vehicle detection.
 - Pedestrian crossing requests.
-

Reactive Decision Rules

The agent uses simple condition-action rules.

IF traffic is HIGH

THEN increase green-light duration

IF traffic is LOW

THEN reduce green-light duration

IF emergency vehicle detected

THEN provide priority signal

IF queue length is HIGH

THEN extend green signal

Streaming Data Processing

Traffic data continuously arrives from sensors and cameras.

A streaming platform can process this information:

Sensors

↓

Kafka / Streaming System

↓

Real-Time Processing

CO5 AT3

↓

Reactive Agent

↓

Traffic Signal Controller

The agent does not need to wait for a complete historical dataset. It reacts to the latest traffic state.

Example

Suppose:

North Road → 80 vehicles

South Road → 20 vehicles

East Road → 15 vehicles

West Road → 10 vehicles

The agent detects heavy traffic on the North Road and increases its green-light duration.

North → GREEN 60 sec

South → GREEN 20 sec

East → GREEN 15 sec

West → GREEN 15 sec

Advantages

- Real-time response.
- Simple implementation.
- Low computational overhead.
- Reduces traffic waiting time.
- Suitable for streaming environments.

Conclusion

A Reactive Agent can continuously monitor traffic conditions and immediately adjust traffic signals. By combining sensors with streaming data processing, the system can respond quickly to congestion and improve traffic flow.

2. GOAL-BASED AGENT FOR HEALTHCARE DIAGNOSIS

Introduction

A **Goal-Based Agent** selects actions based on a desired goal.

In healthcare, the goal may be:

Identify the patient's disease and select an appropriate treatment pathway.

The agent evaluates patient information and selects actions that help achieve the diagnostic goal.

Objective

The system aims to:

- Analyse patient data.
 - Identify possible diseases.
 - Set diagnostic goals.
 - Evaluate possible treatment pathways.
 - Select an appropriate action.
-

System Architecture

Patient

↓

Medical Data

↓

Data Pipeline

↓

Preprocessing

↓

AI Diagnostic Agent

↓

Disease Prediction

↓

Treatment Options

↓

Goal Evaluation

↓

Recommended Pathway

Inputs

The agent can process:

- Age.
 - Symptoms.
 - Blood pressure.
 - Glucose.
 - Laboratory results.
 - Medical history.
 - Previous diagnoses.
-

Diagnostic Goal

Example:

Goal:

Identify whether the patient has diabetes.

The agent evaluates available evidence.

Patient Data

↓

Glucose → High

BMI → High

Family History → Positive

↓

Diabetes Risk

↓

Diagnostic Goal Achieved

Decision-Making Logic

The agent can use:

- Decision Trees.
- Rule-based systems.

- Machine Learning.
- Bayesian reasoning.
- Clinical knowledge bases.

Example:

IF glucose is high

AND BMI is high

AND symptoms are present

THEN increase diabetes risk.

Data Pipeline

Patient Records



Data Cleaning



Feature Extraction



ML Model



Risk Prediction



Goal Evaluation



Treatment Pathway

Example

Suppose the system receives:

Glucose = High

BMI = High

Age = 52

Family History = Yes

The agent evaluates possible outcomes and identifies diabetes as a high-risk possibility.

It may then recommend a pathway such as:

Further Diagnostic Test

↓

Clinical Evaluation

↓

Lifestyle Management

↓

Follow-up Monitoring

The final clinical decision should remain with a qualified healthcare professional.

Advantages

- Goal-directed decision-making.
- Can combine multiple data sources.
- Supports clinical decision-making.
- Can process large patient datasets.
- Provides systematic diagnostic pathways.

Conclusion

A Goal-Based Agent can use patient information and clinical objectives to determine appropriate diagnostic pathways. Data pipelines and AI models allow the agent to process large amounts of medical information efficiently.

3. UTILITY-BASED AGENT FOR E-COMMERCE RECOMMENDATIONS

Introduction

A **Utility-Based Agent** selects an action that maximizes a utility or satisfaction value.

In e-commerce, the agent evaluates multiple products and recommends products that provide the highest expected utility for the customer.

Objective

The system aims to:

- Understand user preferences.
- Evaluate products.
- Assign utility values.

- Generate personalized recommendations.
 - Maximize customer satisfaction.
-

System Architecture

User



Browsing / Purchase History



Large-Scale Data Processing



User Profile



Product Candidate Generation



Utility Calculation



Ranking



Personalized Recommendations

Inputs

The agent can use:

- Previous purchases.
 - Search history.
 - Product ratings.
 - Price preferences.
 - Product categories.
 - Browsing behaviour.
 - Time spent on products.
-

Utility Function

A utility score can be calculated using:

$$[\\begin{aligned} U(p) &= w_1 \\text{Preference} + w_2 \\text{Price} + w_3 \\text{Rating} + w_4 \\text{Quality} \\end{aligned}]$$

where:

- $U(p)$ = utility of product (p)
- (w_1, w_2, w_3, w_4) = importance weights.

Example

Suppose a customer wants a laptop.

The agent evaluates:

Product	Preference	Rating	Price	Utility
---------	------------	--------	-------	---------

Laptop A	0.90	0.85	0.70	0.83
----------	------	------	------	------

Laptop B	0.80	0.95	0.60	0.79
----------	------	------	------	------

Laptop C	0.70	0.80	0.90	0.77
----------	------	------	------	------

The agent recommends:

Laptop A

because it has the highest utility.

Large-Scale Data Processing

An e-commerce company may process millions of records.

Customer Data

↓

Product Data

↓

Transaction Data

↓

Distributed Processing

↓

Feature Generation

↓

Utility Calculation



Recommendation

Technologies such as **Apache Spark/PySpark** can process large datasets efficiently.

Advantages

- Personalized recommendations.
- Considers multiple factors.
- Optimizes customer satisfaction.
- Handles large datasets.
- Can dynamically update recommendations.

Conclusion

A Utility-Based Agent evaluates different product choices using utility functions and selects the option that provides the greatest expected value to the customer.

4. LEARNING AGENT FOR PERSONALIZED EDUCATION SYSTEM

Introduction

A **Learning Agent** improves its performance through experience and feedback.

In an educational platform, the agent observes student performance and continuously adapts learning content and difficulty.

Objective

The system aims to:

- Monitor student performance.
 - Learn from feedback.
 - Adapt content difficulty.
 - Identify weak topics.
 - Personalize learning paths.
-

Learning Agent Architecture

Student



Learning Platform



Student Interaction Data



Distributed Data Processing



Learning Agent



Performance Analysis



Content Recommendation



Student Feedback



Agent Learning

Agent Components

1. Performance Element

Makes decisions about which learning material to provide.

2. Learning Element

Learns from student performance.

3. Critic

Evaluates whether the recommendation was successful.

4. Problem Generator

Suggests new learning activities to improve student performance.

Example

Suppose a student performs:

Easy Questions → 90%

Medium Questions → 70%

Hard Questions → 30%

CO5 AT3

The agent identifies difficulty with advanced questions.

It may generate:

More medium-level practice



Concept explanation



Practice questions



Hard questions

If the student improves, the agent gradually increases the difficulty.

Feedback Loop

Student Attempts Question



Result



Agent Evaluation



Content Difficulty Updated



New Question Selected



Student Attempts Again

PySpark Integration

Large educational platforms can generate huge amounts of data.

Examples:

- Quiz attempts.
- Video watching time.
- Assignment marks.
- Login frequency.

- Learning progress.

PySpark can process this data:

Student Data

↓

PySpark

↓

Distributed Processing

↓

Student Performance Features

↓

Learning Agent

↓

Personalized Content

Advantages

- Personalized education.
- Continuous learning.
- Adaptive difficulty.
- Handles large student datasets.
- Improves learning efficiency.

Conclusion

A Learning Agent continuously learns from student interactions and feedback. By integrating distributed processing systems such as PySpark, it can personalize educational content for large numbers of students.

5. MULTI-AGENT SYSTEM FOR DISASTER MANAGEMENT

Introduction

A **Multi-Agent System (MAS)** consists of multiple autonomous agents that communicate and cooperate to solve a complex problem.

In disaster management, different agents can perform different tasks such as detection, decision-making and communication.

Objective

The system aims to:

- Detect disasters quickly.
- Analyse disaster conditions.
- Coordinate emergency response.
- Share information.
- Support real-time decision-making.

Agents

1. Sensor Agent

Collects information from:

- Temperature sensors.
- Smoke sensors.
- Cameras.
- Water-level sensors.
- Seismic sensors.

2. Decision Agent

Analyses sensor information and determines the severity of the disaster.

3. Communication Agent

Sends alerts to:

- Emergency teams.
- Hospitals.
- Police.
- Fire departments.
- Citizens.

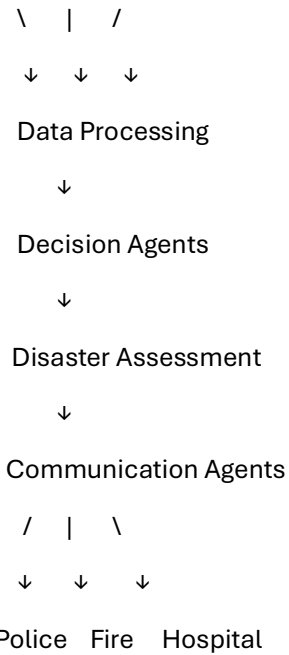
System Architecture

Sensor Agents

/ | \

↓ ↓ ↓

Sensor Camera Satellite



Working

Step 1 – Detection

Sensor agents continuously monitor the environment.

Example:

Smoke Level = Very High

Temperature = High

Step 2 – Analysis

The decision agent analyses the sensor information.

High Smoke + High Temperature



Fire Detected

Step 3 – Coordination

The decision agent communicates with other agents.

Fire Agent → Fire Department

Communication Agent → Public

Medical Agent → Hospital

Step 4 – Response

Emergency teams are assigned based on:

- Location.
- Severity.
- Available resources.
- Distance.
- Number of affected people.

Coordination Strategy

Agents can coordinate using:

- Message passing.
- Shared knowledge bases.
- Event-driven communication.
- Distributed decision-making.

Example:

Sensor Agent

↓

"Fire detected"

↓

Decision Agent

↓

"Severe Fire"

↓

Communication Agent

↓

Emergency Services

Real-Time Data Sharing

The system can use streaming data:

IoT Sensors

↓

Streaming Platform

↓

Real-Time Processing



Agent Communication



Emergency Decision

This allows the system to react rapidly to changing disaster conditions.

Advantages

- Distributed decision-making.
- Faster disaster detection.
- Parallel processing.
- Better resource coordination.
- Fault tolerance.
- Real-time response.

Conclusion

A Multi-Agent System is highly suitable for disaster management because different agents can independently perform sensing, analysis, communication and response tasks while cooperating toward a common objective.

QUICK COMPARISON FOR PRESENTATION

Agent Type	Application	Main Feature	Decision Method
Reactive Agent	Smart Traffic	Immediate response	Condition → Action
Goal-Based Agent	Healthcare	Achieves diagnostic goal	Goal-oriented search
Utility-Based Agent	E-Commerce	Maximizes satisfaction	Highest utility
Learning Agent	Education	Improves with experience	Feedback and learning
Multi-Agent System	Disaster Management	Agent collaboration	Distributed coordination

Easy Way to Remember

Reactive → "What is happening now?"

Goal-Based → "What goal should I achieve?"

CO5 AT3

Utility-Based → "Which option is best?"

Learning → "How can I improve?"

Multi-Agent → "How can multiple agents work together?"

These five examples collectively cover the major **AI agent architectures: Reactive, Goal-Based, Utility-Based, Learning, and Multi-Agent systems.**